

CSA1701-ARTIFICIAL INTELLIGENCE

LAB EXPERIMENTS

1. 8-Puzzle Problem

Aim: To develop a Python program that solves the 8-Puzzle Problem using a search algorithm (such as Breadth-First Search - BFS) to find the shortest sequence of moves from the initial state to the goal state.

Algorithm:

1. Represent the puzzle as a 3x3 matrix.
2. Define the goal state.
3. Locate the empty tile (0).
4. Generate all possible moves (up, down, left, right).
5. Use Breadth-First Search (BFS) to explore possible states.
6. Keep track of visited states to avoid repeated exploration.
7. If the goal state is reached, display the solution path.
8. If all possibilities are exhausted, report that no solution exists.

```

# S PUZZLE.py
from collections import deque

GOAL = (
    (1, 2, 3),
    (4, 5, 6),
    (7, 8, 0)
)

def get_neighbors(state):
    neighbors = []
    # Find the blank tile (0)
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                x, y = i, j
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    for dx, dy in directions:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [list(row) for row in state]
            new_state[x][y], new_state[nx][ny] = (
                new_state[nx][ny],
                new_state[x][y],
            )
            neighbors.append(tuple(tuple(row) for row in new_state))
    return neighbors

def bfs(start):
    queue = deque([(start, [])])
    visited = set()

    while queue:
        state, path = queue.popleft()
        if state == GOAL:
            return path + [state]
        if state in visited:
            continue
        visited.add(state)

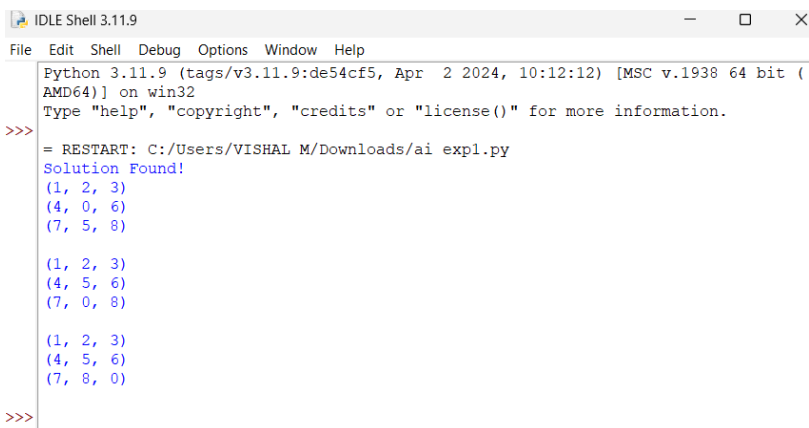
        for neighbor in get_neighbors(state):
            queue.append((neighbor, path + [state]))
    return None

start_state = (
    (1, 2, 3),
    (4, 0, 6),
    (7, 5, 8)
)

solution = bfs(start_state)
if solution:
    print("Solution Found!")

```

Output:



```

IDLE Shell 3.11.9
File Edit Shell Debug Options Window Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> = RESTART: C:/Users/VISHAL M/Downloads/ai expl.py
Solution Found!
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
>>>

```

Result: Thus, the 8-Puzzle Problem solved successfully using Python.